

Lecture 4: PKI + Channel Security

4.1 Distribution of public key

4.2 PKI

4.2.1 Certificate

4.2.2 CA

4.3 Limitations/attacks on PKI

4.4 Strong authentication

4.5 Key-exchange

4.6 Putting all together: Securing Communication Channel

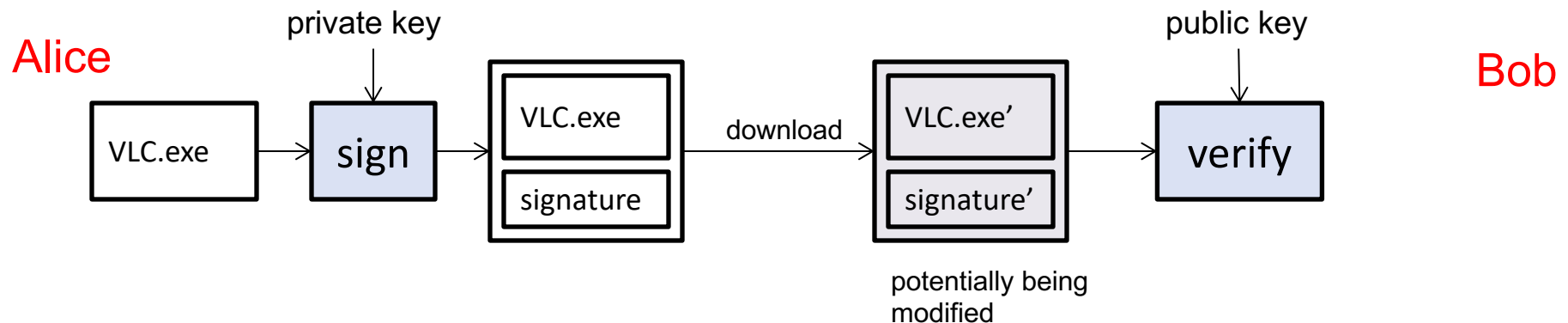
Summary & takeaways

- We need a “secure broadcast channel” to distribute public keys.
 - Certificate: a piece of document stating the binding of a public key to a name. The document is signed (certified) by a CA. It allows offline verification of public key.
 - CA’s chain of trust. PKI: the whole infrastructure. Limitations: Too many CA’s.
 - PKI distributes associations of (name, public key). Social engineering exploits confusing naming convention, in particular domain name, to trick user.
- Authentication protocols
 - Unilateral, mutual. Adversary sniffs previous interactions and wants to impersonate. (no key exchange. Assuming each entity remains the same throughout the session.)
 - DH key exchange: Adversary (Eve) sniff pervious and current interactions and wants to guess the key. No authentication.
 - Authenticated key exchange. Adversary is Mallory (can sniff, spoof, interrupt the session) (PKC based, DH based).
- Putting all together. With crypto primitives, we can obtain a secure (w.r.t. authenticity & confidentiality) channel on top of an underlying public channel.
 - (1) Use long term key for Authenticated key exchange to get session key (2) Session key to protect confidentiality (encrypt) & integrity (mac) of subsequent messages.
 - Authenticated encryption.
 - Why session key: More efficient. Forward secrecy. Possible to segregate authentication (long term key) and communication (using session key) as two components.

4.1 Public key Distribution

Overview: examples of signature application

- Consider the previous example on VLC.exe.
- To overcome the need of a secure channel to send the unkeyed digest or the symmetric key, we can use signature (public key) in this way:
 1. The developer, say Alice, “signed” the file VLC.exe using the the developer’s private key.
 2. A user, say Bob, who has downloaded the file VLC.exe (together with the signature) from an unverified source (for e.g. from the CNET download site) , can verify the authenticity of the file using Alice’s public key.



Secure Channel:

Compared to the symmetric key setting (mac), here, we don't need to have a secret key between Alice to Bob.

However, we still need a secure channel for Alice to send her public key to Bob.

Example of “signed” email using PGP public key

- Alice (with email account `alice@comp.nus.edu.sg`) sent an email to Bob. Alice has a pair of “PGP” public-private key. Alice’s email is signed using her private key. (see next slide for the actual email sent).
- After Bob has received the email, with Alice’s public key, he can check the authenticity by verifying the signature.
- To carry out the authenticity, Bob needs to know Alice’s public key.
- We are now back to square one: we need some secure channel for Alice to send her public key to Bob.

Example of “signed” email using PGP public key

Date: Wed, 07 Mar 2007 03:22:08 +0800
From: Alice Ho <alice@comp.nus.edu.sg>
User-Agent: Thunderbird 1.5.0.10 (Windows/20070221)
MIME-Version: 1.0
To: bob@comp.nus.edu.sg
Subject: My first signed email
X-Enigmail-Version: 0.94.2.0
Content-Type: text/plain; charset=ISO-8859-1
Content-Transfer-Encoding: 7bit

This part is not signed,
i.e. not included in
computing the signature

-----BEGIN PGP SIGNED MESSAGE-----

Hash: SHA1

Dear Bob,

This is my very first signed email and I want you to keep it =)

Regards,

Alice Ho

-----BEGIN PGP SIGNATURE-----

Version: GnuPG v1.4.3 (MingW32)

Comment: Using GnuPG with Mozilla - <http://enigmail.mozdev.org>

iD8DBQFF7b9XMJcr5kFKO4IRAk+yAKC7JVIleY+aHEAqqCeVdYGOE10PmwCg9DrE

ArgWymKbDnl7m9W1leVeQqM=

=EksE

-----END PGP SIGNATURE-----

Message

The signature

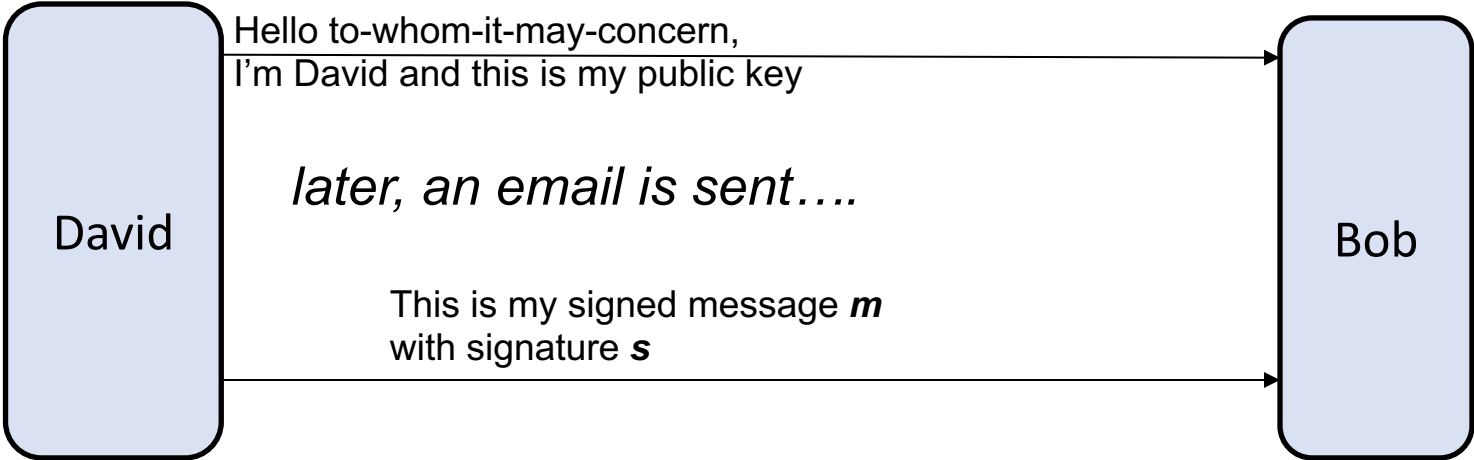
What if public key is spoofed?

David pinned his namecard on signboard

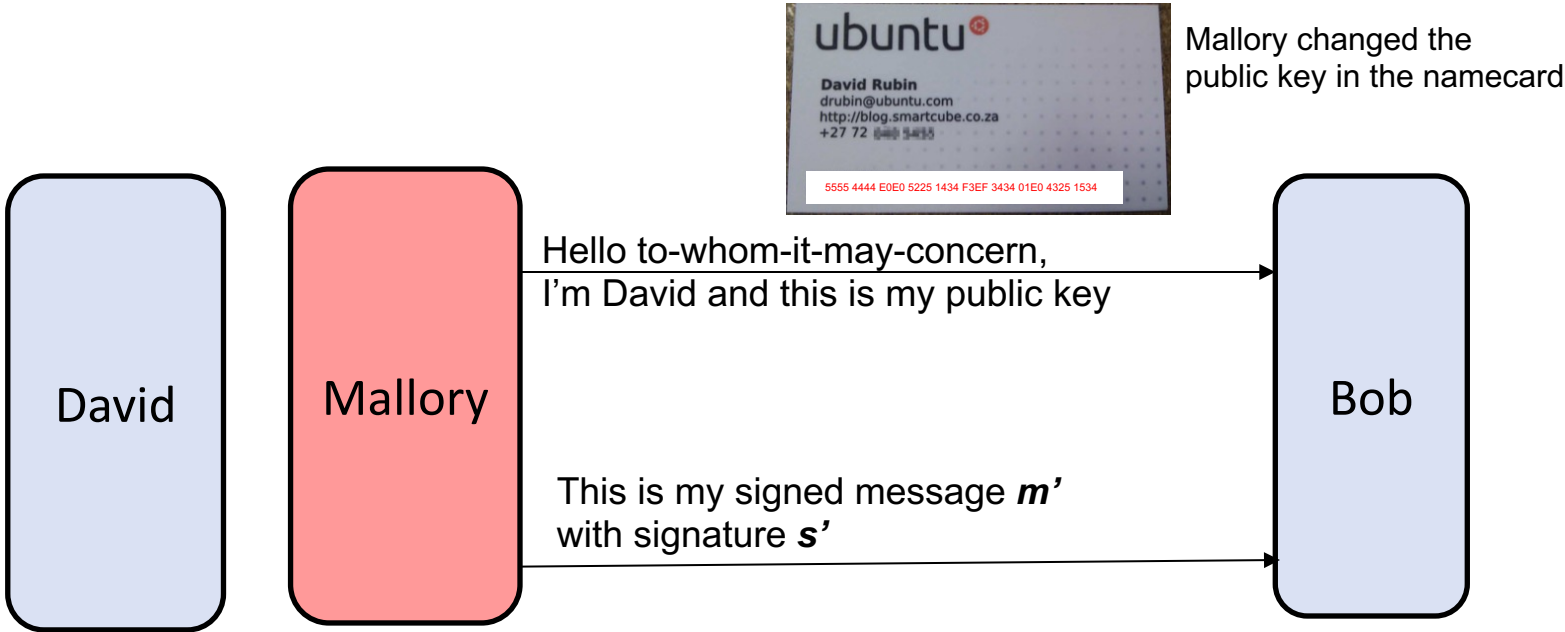


Image from <https://wordtothewise.com/2014/09/alice-and-bob-and-pgp-keys/>

Intended situation



Under attack



Mallory changed the public key in the namecard.

Public key distribution vs Symmetric key distribution

Still need a secure channel. Nevertheless, it is easier to securely “distribute/broadcast” public key compared to “establish” different symmetric key for each pair.

- **Public key:** An entity broadcast its public key to some public billboard only once. (linear)
- **Symmetric key:** An entity needs to securely establish a different symmetric key with each of the rest. (quadratic)
- **Public key:** An entity doesn't need to know the existence of the receiver while broadcasting its public key. (In the previous slide, David could place his name card in canteen's table, stick in signboard and doesn't need to interact with Bob)
- **Symmetric key:** Both entities interact to establish the key.

Key distribution

The previous example illustrates the need of a mechanism to distribute public keys. With the public key securely distributed, we can use it for encryption (*confidentiality*) and signature verification (*authenticity*). In practice, mostly on authentication.

3 methods:

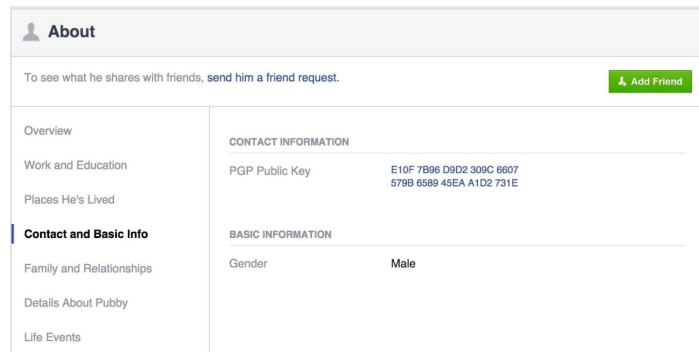
- Public Announcement on existing mechanism (Social media, name-card)
- Publishing publicly available directory.
- Public Key Infrastructure.

Key distribution: Public Announcement

- The owner broadcasts her public key. For example, by sending it to friends via email, publishing in social media, or simply the physical namecard.

Many owners listed their “PGP public key” in blog, personal webpage, etc.

Facebook



<https://wordtothewise.com/2014/09/alice-and-bob-and-gpg-keys/>

**:I saw the above in Facebook's help-page but can't add it in my account.*

- Limitations:
 - Not standardized.
 - No systematic way to search/verify the public key.

(optional) Get a PGP public key for yourself and send a signed email.

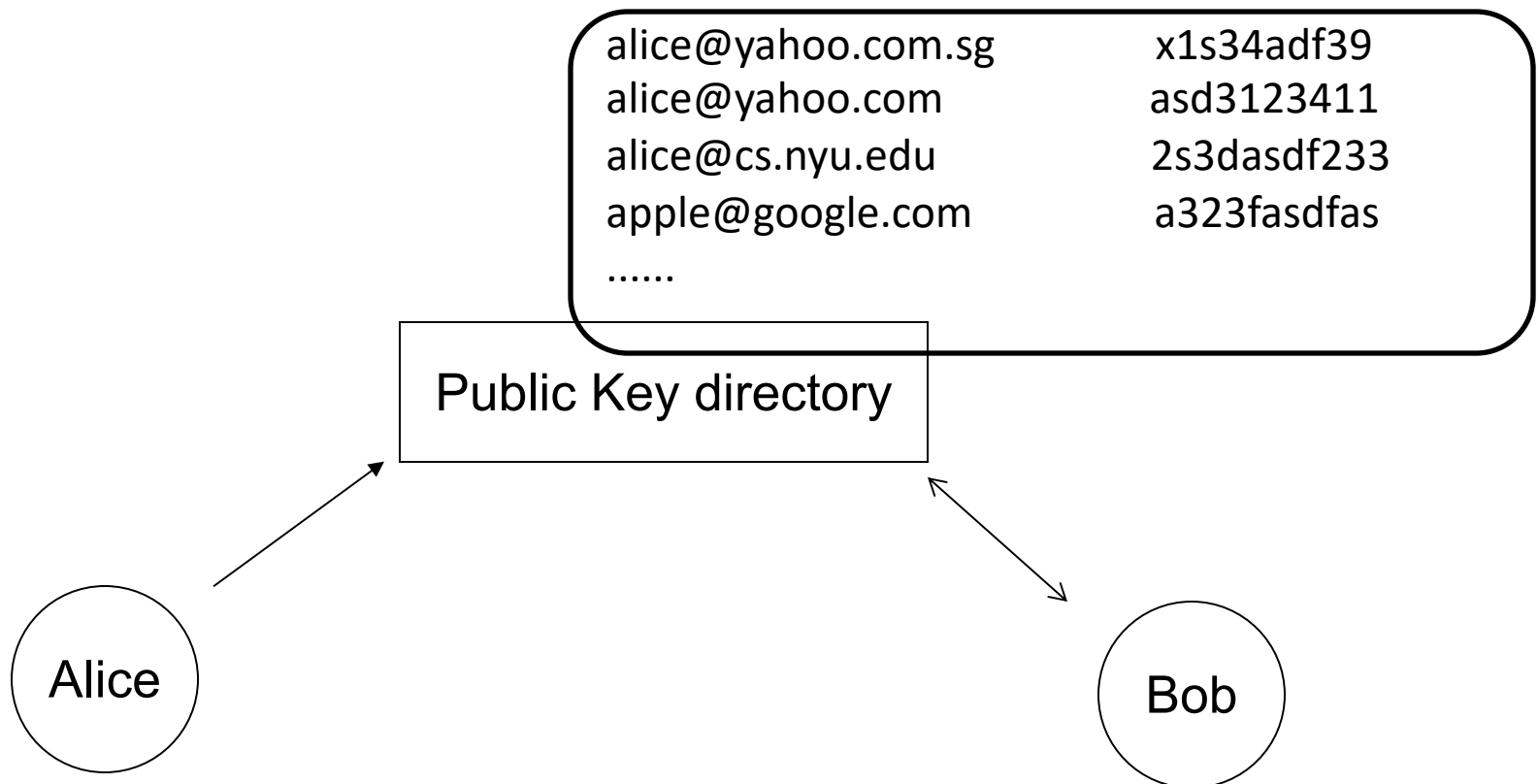
Key distribution: Publicly Available Directory

If Bob wants to find the “**public key**” associated to a “**name**”, say

alice@yahoo.com.sg

he can search the public directory by querying the directory server.

For Example: <https://pgp.mit.edu/>



Potential Issues:

- Anyone can post their public keys in the server. Not clear how to verify the info. Suppose the server receives a request to post a public key, how does the server verify that the information is authentic? (for email, the server could verify by sending a confirmation email/OTP to the address. What about other types of name, such as web domain name, passport number?)
- In term of performance, the server is the bottleneck and is the “single-point-of-failure”.
- Not everyone trust the server. E.g. Certain entities might not trust mit.edu to host the server.

PKI

- PKI is a standardized system that distribute public keys. To address limitation of previous 2 methods. A main objective is to be **deployable** on a large scale.
- Centered around two components/notions:
 - ***Certificate***
 - ***Chain of trust of Certificate Authority (CA)***
- Other related issues: ***Revocation***.

4.2 Public Key Infrastructure

- 4.2.1. Certificate & CA
- 4.2.2. CA's chain-of-trust
- 4.2.3. Revocation

4.2.1 Certificate & CA

Certificate Authority (CA)

- “**Certificates**” are useful in distributing public keys. An CA issues certificates. (next few slides)
- Essentially, CA is a trusted authority that manages a directory of public keys. An entity can request adding its public key to the public directory. Anyone can send queries to search the directory. (Certificates facilitate checking/querying to be done in an offline manner.)
- The CA also has its own public-private key. We first assume that the CA’s public key has been securely distributed to all entities involved. (so, we still need a secure channel to distribute the CA’s public key).
- Most OSes and browsers have a few pre-loaded CAs' public keys: they are known as the “**root**” CAs. Not all CAs’ public keys are preloaded. Other CA’s public keys can be added through the chain-of-trust to be described later.
- When Bob’s machine has a CA’s public key, we sometime say that the Bob has “installed” the CA’s public key/root-certificate.
- While a browser and system providers can pre-load any root CA’s public key of their choices, there are well-accepted stringent requirements. For e.g, it must pass **WebTrust audit** (<http://www.webtrust.org/homepage-documents/item76002.pdf>)

Certificate

Consider the situation where Bob wants to get/verify Alice's public key. One method is for Bob to query the CA.

There are two limitations of querying the CA as-and-when needed:

- The CA became a bottleneck
- Bob needs to have online access to the CA at the point of verification

Using certificates is a “smart” way to check the public key in an offline manner.

Bob getting Alice's public key: Without Certificate

We assume that Bob has the public key of CA. Hence the authenticity of the messages from CA (i.e. Step 3) can be verified: CA signs its message.

alice@yahoo.com.sg	x1s34adf39
alice@yahoo.com	asd3123411
alice@cs.nyu.edu	2s3dasdf233
apple@google.com	a323fasdfas
.....	

Directory Server (CA)

(Step 2) Bob: What is the public key of alice@yahoo.com.sg?

(Step 3) CA: The public key of alice@yahoo.com.sg is x1s34adf39 and it is valid until 1 Sep 2019. (Signed by CA)

Alice

Bob

(Step 1) Alice: This is an email from alice@yahoo.com.sg. The email is “signed” using my private key. My public key is x1s34adf39. You can contact CA to verify that I am not lying.

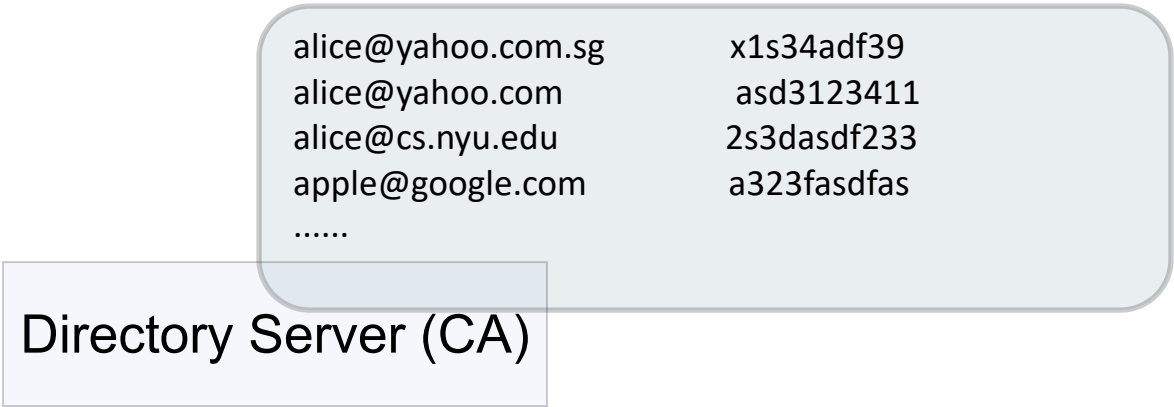
From: alice@yahoo.com.sg
Subject: Hello Bob
Meeting 3pm at the usual place today.

My public key is: x1s34adf39
signature: xsdewsd

You can check with CA.

With Certificate

Note that in the previous slide, data in step 3 is always the same. So, a “lazy” CA would “sign” the message before hand and pass it to Alice. Alice can directly send the signed message to bob. That signed message is called the “certificate”.



(Step 2) Bob verifies that the signature in the certificate is indeed signed by the CA. Since no one except the CA can produce the valid signature, the authenticity of the information in the certificate is as good as coming directly from the CA.



Certificate

- A **certificate** is a digital document that contains **at least** the following 4 main items
 - 1) The name, for e.g. *alice@yahoo.com* or *bbc.com* or **.bbc.com*
 - 2) The public key of the owner.
 - 3) The time window that this certificate is valid.
 - 4) The signature of the CA.
- Another important information:
 - Usage of certification: E.g. (1) the type of “name”, whether it is email address or domain name. (2) whether the “name” can take the role of a CA (chain-of-trust).
- Other info
 - Digest (Fingerprint). For verification without using CA’s public key (e.g. the VLC.exe setting in earlier slides)
 - Meta data such as type of algorithm (whether ECC or RSA, key length etc)

- Now, from the certificate, Bob can obtain Alice's public key, and verifies its authenticity even without connection to the CA.
- Bob trusts the CA. Hence, information signed/certified by the CA is also trusted.
- In other words: We first assume that Bob has the authentic CA's public key. Thus, information verified using CA's public key is authentic. Hence, the public key extracted from the certificate is indeed Alice's. From Alice's public key, Bob can verify authenticity of the email.

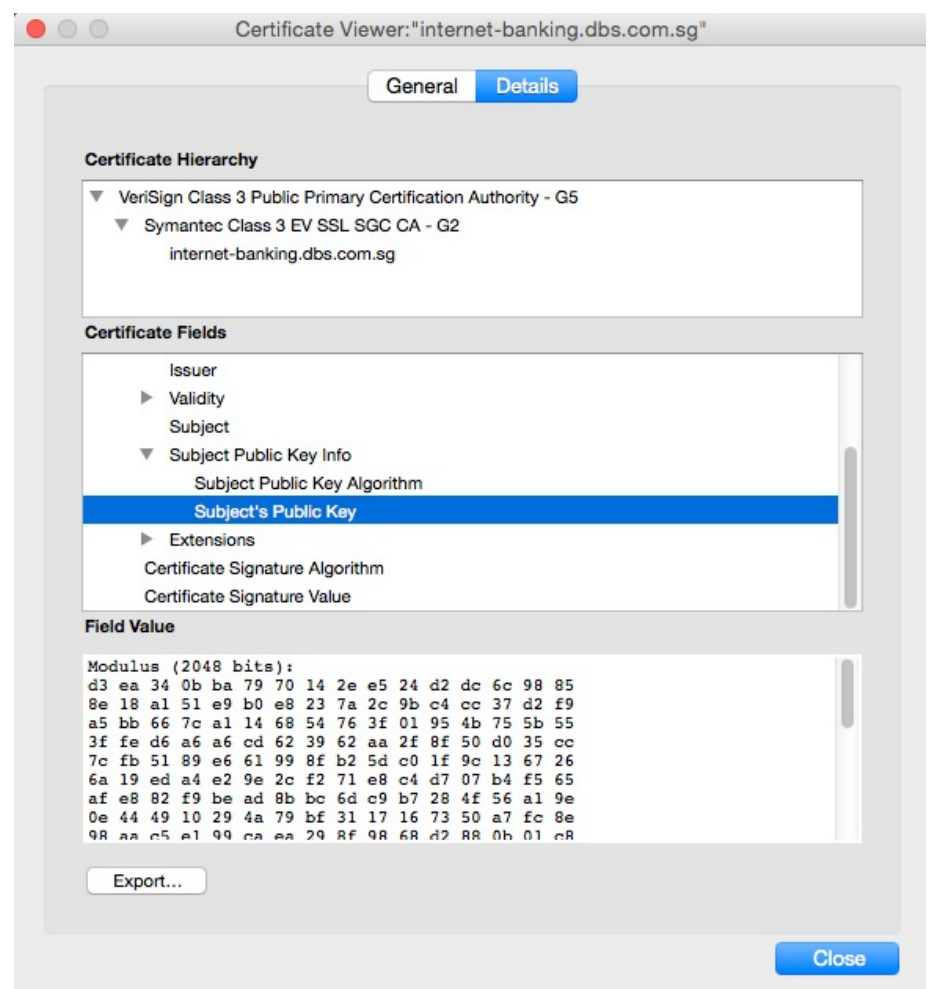
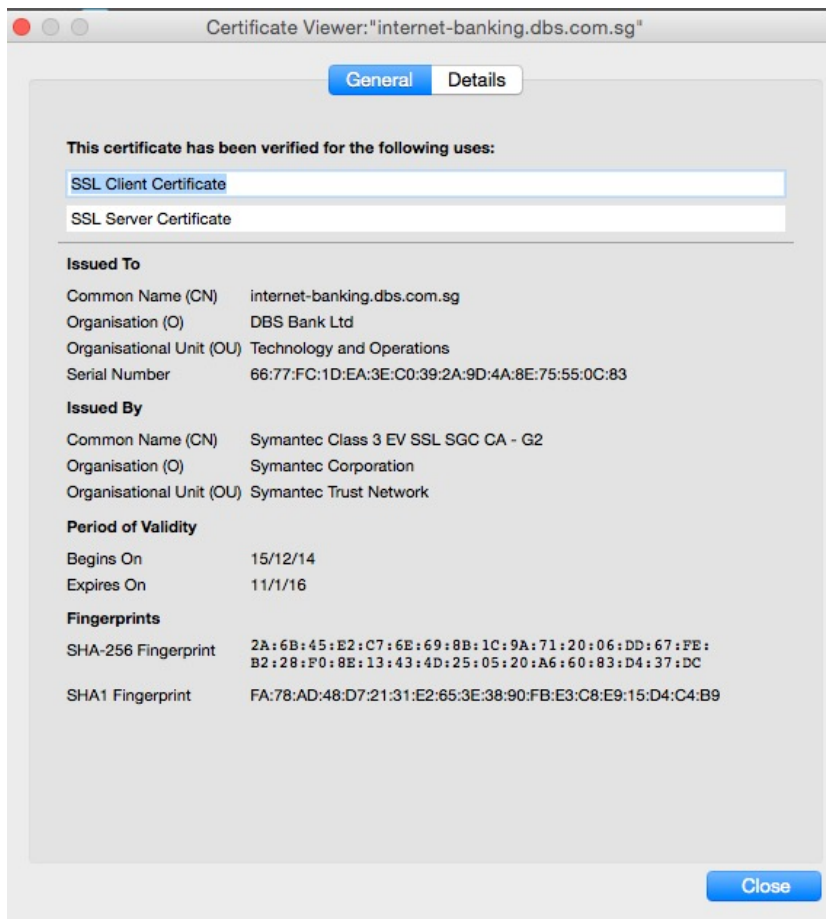
- (1) Bob trusts the CA's public key in his machine.
- (2) CA's public key verifies that the certificate is authentic and thus Alice's public key is correct.
- (3) From Alice's public key, Bob verifies that the email is authentic.

Example of Certificate

Visit <https://internet-banking.dbs.com.sg/IB/Welcome>

See the certificate's detail. (click the address bar)





Standard: X509

Standardization **bodies**:

- ITU-T X.509:
Specifies formats for certificates, certificate revocation lists, and a certification path validation algorithm
- The Public-Key Infrastructure (X.509) Working Group (PKIX):
IETF working group that creates Internet standards on issues related PKI based on X.509 certificates

What the standard standardize?

Eg:

- **Structure** of an X.509 v3 digital certificate:
 - Certificate:
 - Version Number
 - Serial Number
 - Signature Algorithm ID (*Note Signature Algorithm below too*)
 - Issuer Name
 - Subject Name
 - Subject Public Key Info: Public Key Algorithm, Subject Public Key
 - Validity period: Not Before, Not After
 - Issuer Unique Identifier (optional)
 - Subject Unique Identifier (optional)
 - Extensions (optional)
 - Certificate Signature Algorithm
 - Certificate Signature

How Do I Get a Certificate?

- Goto a **root CA** to issue you one:
 - Paid ones: \$10 - \$50 / year (not costly)
- “*Let's Encrypt*” provides (basic) TLS certs at **no charge**:
 - Launched in April 2016
 - A certificate is valid for 90 days
 - Its renewal can take place at anytime
 - Automated process of cert creation, validation, signing, installation, and renewal
 - No of issued certs: **1M** (March 2016) to **380M** (Sept 2018)
- Firefox Telemetry:
 - **77%** of all page loads via Firefox are now encrypted
 - It was predicted that it will reach **90%** by the end of 2019

Self-signed certificate

- See tutorial.
- A self-signed certificate is not signed by a CA. It is signed by the “name” in the certificate. The certificate is to be verified using that “public key” in the certificate.
- Self-signed certificate, although sounds contradicting and useless, is convenient in manual installation of public key. If a user wants to manually install a public key, the user can download the “self-signed certificate” and manually accept the certificate. Essentially, by accepting the self-signed certificate, the user instructs the machine to accept the binding of the “name” and the “public key”, although the certificate is not signed by a CA.
- A self-signed certificate by a CA is sometime called a “root-certificate” (always take extra caution in accepting root-certificate).

Summary

- A certificate is simply a document signed by a CA.
 - (1) An identity.
 - (2) The associated public key
 - (3) The time window that this certificate is valid.
 - (4) The signature of the CA.
- The document “certifies” that the public key is indeed belonged to the claimed identity.
- We assume that Bob already has CA’s public key “installed” in his machine.

4.2.2 Certificate Authority & Trust relationship

Responsibility of CA

- The CA, besides issuing certificate, is also responsible to verify that the information is correct. For instance, if someone wants request for a certificate for the identity

www.nus.edu.sg

the CA should check that the applicant indeed own the above domain name. This may involve manual checking and thus it could be costly.

Getting a certificate signed by a CA is not free.

Certificate Chain-of-trust

- There are many CA's.
- Most OS, browsers already have a few CA's public key pre-loaded. These are the “**root CA**”.
- Suppose Alice's certificate is **issued** (i.e. signed) by CA#1, but Bob doesn't have the public key of CA#1, why should he do?
- In the first place, Alice, anticipating the Bob might not have the public key of CA#1, can send her email, her certificate, and CA#1 certificate (issued by the root CA) to Bob. Bob can now
 - verify CA#1's certificate using root CA's public key.
 - verify Alice's certificate using CA#1's public key.
 - verify Alice's email using Alice's public key.

From: alice@yahoo.com.sg
Subject: Hello Bob
Meeting 3pm at the usual
place today.

signature: xsdewsd

- name : alice@yahoo.com.sg
- public key: x1s34adf39
- Valid until: 1 Sep 2019
- signature of CA#1

- name :CA#1
- public key: x3141342
- Valid until: 1 Sep 2020
- note: *CA#1 is a Certificate Authority and thus can issue certificate*
- signature of Root CA

In our example, the certificate issued to CA#1 clearly indicate that CA#1 is a certificate authority and can issue certificate. Without that “note”, the owner can't issue other certificates.

- If Alice doesn't attach CA#1's certificate, then Bob has to obtain it from other sources.

Example

Check the certificate of www.dbs.com.sg. Most browsers would show the chain. (below obtained from Firefox)

DBS's certificate, signed by **Entrust CA-L1M**, signed by **Entrust Root CA-G2**.

Certificate

www.dbs.com.sg	Entrust Certification Authority - L1M	Entrust Root Certification Authority - G2
Subject Name		
Country	SG	
Locality	Singapore	
Inc. Country	SG	
Organization	DBS Bank Ltd	
Business Category	Private Organization	
Serial Number	196800306E	
Common Name	www.dbs.com.sg	
Issuer Name		
Country	US	
Organization	Entrust, Inc.	
Organizational Unit	See www.entrust.net/legal-terms	
Organizational Unit	(c) 2014 Entrust, Inc. - for authorized use only	
Common Name	Entrust Certification Authority - L1M	
Validity		
Not Before	Wed, 08 Sep 2021 13:14:55 GMT	
Not After	Fri, 07 Oct 2022 13:14:54 GMT	
Subject Alt Names		
DNS Name	www.dbs.com.sg	
DNS Name	cug.www.dbs.com.sg	
DNS Name	beta.dbs.com.sg	

www.dbs.com.sg	Entrust Certification Authority - L1M	Entrust Root Certification Authority - G2
Subject Name		
Country	US	
Organization	Entrust, Inc.	
Organizational Unit	See www.entrust.net/legal-terms	
Organizational Unit	(c) 2014 Entrust, Inc. - for authorized use only	
Common Name	Entrust Certification Authority - L1M	
Issuer Name		
Country	US	
Organization	Entrust, Inc.	
Organizational Unit	See www.entrust.net/legal-terms	
Organizational Unit	(c) 2009 Entrust, Inc. - for authorized use only	
Common Name	Entrust Root Certification Authority - G2	
Validity		
Not Before	Mon, 15 Dec 2014 16:26:03 GMT	
Not After	Tue, 15 Oct 2030 16:55:03 GMT	
Public Key Info		
Algorithm	RSA	
Key Size	2048	
Exponent	65537	
Modulus	D0B1C13923C2B1D1ECF757DD55243691202248F7FC CA52...	

www.dbs.com.sg	Entrust Certification Authority - L1M	Entrust Root Certification Authority - G2
Subject Name		
Country	US	
Organization	Entrust, Inc.	
Organizational Unit	See www.entrust.net/legal-terms	
Organizational Unit	(c) 2009 Entrust, Inc. - for authorized use only	
Common Name	Entrust Root Certification Authority - G2	
Issuer Name		
Country	US	
Organization	Entrust, Inc.	
Organizational Unit	See www.entrust.net/legal-terms	
Organizational Unit	(c) 2009 Entrust, Inc. - for authorized use only	
Common Name	Entrust Root Certification Authority - G2	
Validity		
Not Before	Tue, 07 Jul 2009 17:25:54 GMT	
Not After	Sat, 07 Dec 2030 17:55:54 GMT	
Public Key Info		
Algorithm	RSA	
Key Size	2048	

Pre-installed,
Self-signed!

Look at the field “Basic Constraint” and “key usage”. It indicates whether the “name” can take the role of CA.

Basic Constraints	
Certificate Authority	No
Key Usages	
Purposes	Digital Signature, Key Encipherment
Extended Key Usages	
Purposes	Server Authentication, Client Authentication

Basic Constraints	
Certificate Authority	Yes
Key Usages	
Purposes	Certificate Signing, CRL Signing
Extended Key Usages	
Purposes	Client Authentication, Server Authentication

Basic Constraints	
Certificate Authority	Yes
Key Usages	
Purposes	Certificate Signing, CRL Signing

Question

- Occasionally, while surfing the web, you may encounter this warning message:
www.example.com uses an invalid security certificate. The certificate is not trusted because the issuer certificate is unknown.
option 1: get me out of here.
option 2: I know the risk. Accept the certificate.

What is going on here? (The “issuer” here probably refers to the CA. So, the browser doesn’t have the CA’s public key.)

- While installing a new package using package manager (this applied to MAC OS, linux, cgywin, etc), say apt-get, you may also encounter similar message:
Packages server certificate verification failed.

What is going on here? (error message indicate failure to verify the certificate but does not give sufficient info on which part fails. It could be certificate expired, no certificate, wrong signature, etc)

4.2.3 Revocation

Certificate Revocation

- Non-expired certificates to be revoked due to different reasons:
 - Private key was compromise
 - Entity left an organization
 - Business entity closed
 - Issuing CA was compromise
- A verifier needs to check whether a certificate in question is *still valid*, although the certificate is not expired yet. (circular/contradicting requirement. Main purpose of certificate is to facilitate offline checking, and yet need to online verify whether it is revoked).
- Two different approaches to certificate revocation:
 - Certificate Revocation List (CRL) :
CA periodically signs and publishes a revocation list
 - Online Certificate Status Protocol (OCSP):
OCSP Responder validates a cert in question
- Either way, either an *online* CRL Distribution Point or an online OCSP Responder is needed.
- Recommendation is for a user (e.g. browser) to periodically (say weekly) update its local cache of revocation list. The user does not need to online check whenever the user wants to verify a certificate.

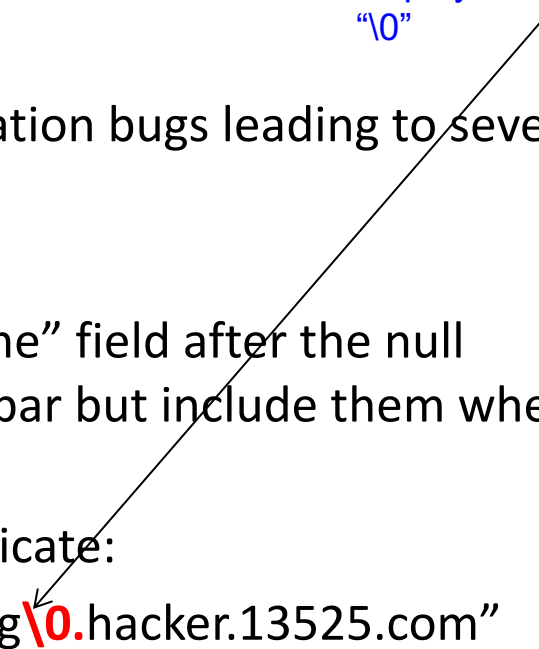
Another Issue: Certificate Revocation

- Mozilla have announced they are deprecating CRL in favor of OCSP.
- There are many other issues.
(optional, see OCSP Stapling and issue on *soft-fail*).

4.3 Limitations/Attacks on PKI

Implementation Bugs (revisit in software security)

the null character is
displayed as the string
“\0”



- There are numerous well-known implementation bugs leading to severe vulnerability. Here is one example:
- Some browsers ignore substrings in the “name” field after the null characters when displaying it in the address bar but include them when verifying the certificate.

(a) Name appeared in the certificate:

“www.comp.nus.edu.sg\0.hacker.13525.com”

or

“*.hacker.13525.com”

(b) The browser displays it as

“www.comp.nus.edu.sg”

- As a result, the viewers thought that they are connecting via https to (b), but in fact is connecting to (a).
- See

www.ruby-lang.org/en/news/2013/06/27/hostname-check-bypassing-vulnerability-in-openssl-client-cve-2013-4073/

Abuse by CA

- There are so many CA's. One of them could be malicious. A rogue CA can practically forge any certificate. Here is a well-known incident.
- Trustwave issued a “subordinate root certificate” (i.e. the receipt can now issue certificate) to an organization for monitoring the network. With this certificate, the organization can “spoof” X.509 certificates and hence is able to act as the man-in-the-middle of any SSL/TLS connection.
- see
ComputerWorld, *Trustwave admits issuing man-in-the-middle digital certificate; Mozilla debates punishment*, Feb 8 2012.

see

<http://www.computerworld.com/article/2501291/internet/trustwave-admits-issuing-man-in-the-middle-digital-certificate--mozilla-debates-punishment.html>

Another famous case of abuse (or ignorant?)

- Lenovo's SuperFish scandal.

(previously reserved as class presentation. In this semester, we would not cover it. This is a good example, but we have too many materials to cover.)

Social Engineering

PKI “certifies” the association (name, public key). In the first place, how the verifiers know that the “name” is correct? Social engineering exploits confusing naming convention, so that the verifiers are tricked into using a wrong name. For web, an attacker first rightfully register for a domain name that resemble a targeted domain name. Next, use the registered domain to confuse the victim in phishing attack. The techniques include “Domain typosquatting”, “Homograph attack” or using sub-domain name. They are aka Domain spoofing, URL spoofing, Fake URL, etc. E.g.

Method 1 (typosquatting)

1. An attacker registered for a domain name
luminus.nus.edv.sg
and obtained a valid certificate of the above name. No one has registered edv.sg and thus the attacker is about to get it.
2. The attacker employed “phishing attack”, tricking the victim to click on the above link, which was a spoofed site of
luminus.nus.edv.sg
3. The address bar of the victim’s browser correctly displayed
<https://luminus.nus.edv.sg>
but the victim didn’t notice that and logged in using the victim’s password.

read <http://en.wikipedia.org/wiki/Typosquatting>

See https://en.wikipedia.org/wiki/IDN_homograph_attack

Method 2 (sub-domain)

A more commonly deployed method uses sub domain. E.g.

Attacker is the rightful owner of 134566.com. Attacker creates a sub.domain

luminus.nus.edu.sg.134566.com.

Since attacker is the owner of 134566.com, it can get a valid certificate of

luminus.nus.edu.sg.134566.com or *.134566.com

Question

- How browsers such as Safari, Firefox, Chrome display the domain name in the address bar?

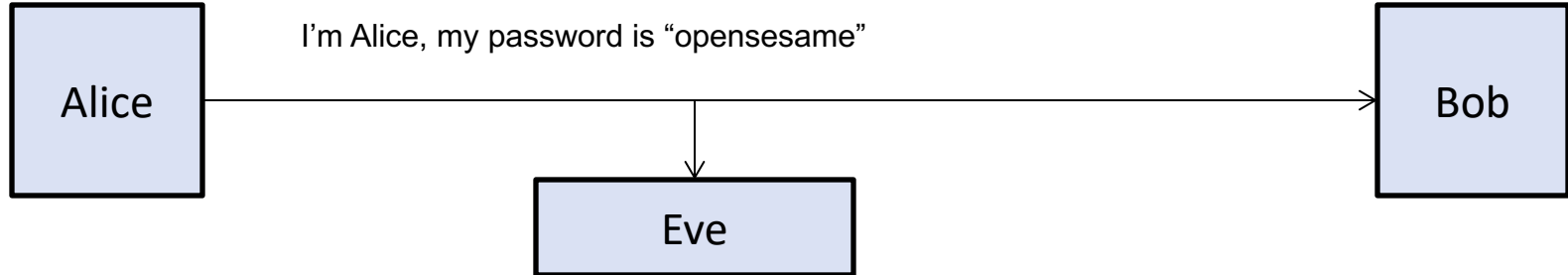
4.4 Authentication

- Authentication using symmetric key
- Authentication using Public key
- We need something more: Authenticated key-exchange.

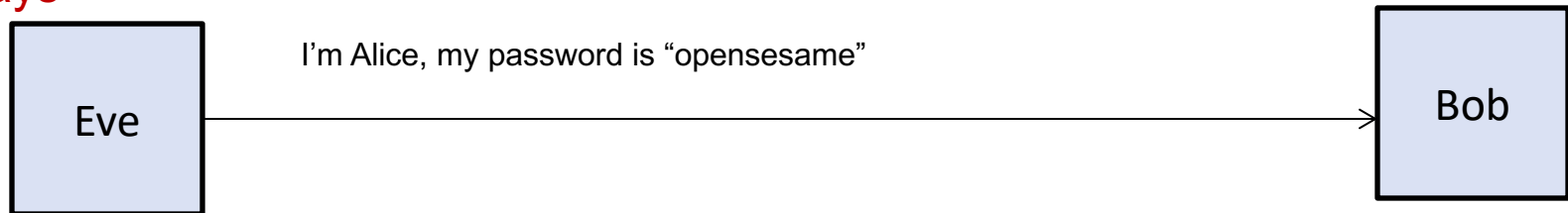
Weak Authentication

Recap that in authentication, an entity who can convince other that it knows a secret is “authentic”. Sending password over is a simple method. But an eavesdropper who get the password can simply replay it.

Eavesdrops



Replays



Interestingly, it is possible to have a mechanism where Alice can “prove” to Bob that she knows the secret, without revealing the secret.

Authentication: Challenge-response

Suppose Alice and Bob have a shared secret k , and both have agreed on a message authentication code. An entity who knows k is either Alice or Bob. Now, an entity P wants to convince Alice that it is Bob.

- (1) P sends to Alice a hello message
“Hi, I’m Bob”
- (2) (Challenge) Alice randomly picks a message m and sends m to P .
- (3) (Response) P computes $t = \text{mac}_k(m)$. P sends t to Alice.
- (4) Alice verifies that the tag received is indeed the mac of m . If so, accepts, otherwise rejects.

Remarks:

- We assume that an attacker first sniff all interactions between Alice and Bob. After that, the attacker communicate with Alice and try to trick Alice to believe that it knows the secret key k .

- By property of mac, even if Eve can sniff the communication between Alice and Bob, and obtain multiple pairs of valid m, t , Eve still (1) can't get the secret key k , and (2) can't forge the mac for messages that Eve has not seen before.
- Eve also can't replay the response. This is because the challenge is randomly chosen and likely to be different in the next authentication session. The challenge m ensures ***freshness*** of the authentication process. It is also known as the *cryptographic nonce* (or simply *nonce*).
- This protocol only authenticates Bob. That is, authenticity of Bob is verified. Hence it is call ***unilateral authentication***. There are also protocols to verify both parties, which are called ***mutual authentication***.

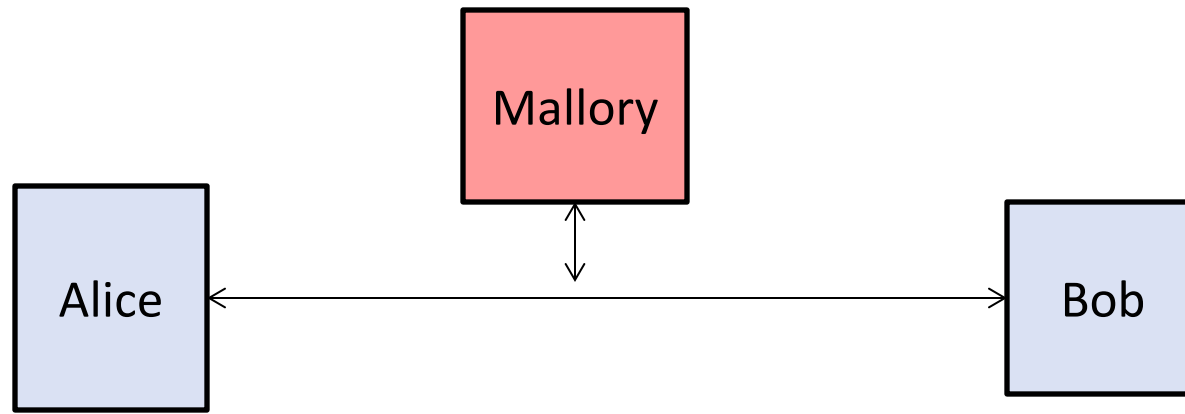
Unilateral authentication using PKC

We can also have a public key version using signature. Suppose Alice wants to authenticate an entity **P** who claims to be Bob.

- (1) (Challenge) Alice chooses a random message r and sends to **P**:
 $\langle \text{"Bob, here is your challenge"}, r \rangle$
 - (2) (Response) **P** uses his private key to sign r . **P** also attaches a certificate
 $\langle \text{sign}(r), \text{certificate} \rangle$
 - (3) Alice verifies the certificate indeed belong to Bob and is valid. Next, extracts the public key from the certificate and verifies that the signature $\text{sign}(r)$ is correct. If so, accept.
- If Alice already knows Bob's public key, the certificate can be omitted.
 - Similarly, we assume an attacker can observe multiple interaction between Alice and the real Bob. After that, the attacker try to convince Alice that it knows the private key. By security property of signature, the eavesdropper can't derive Bob's private key and can't forge the response.
 - The nonce r ensure freshness.

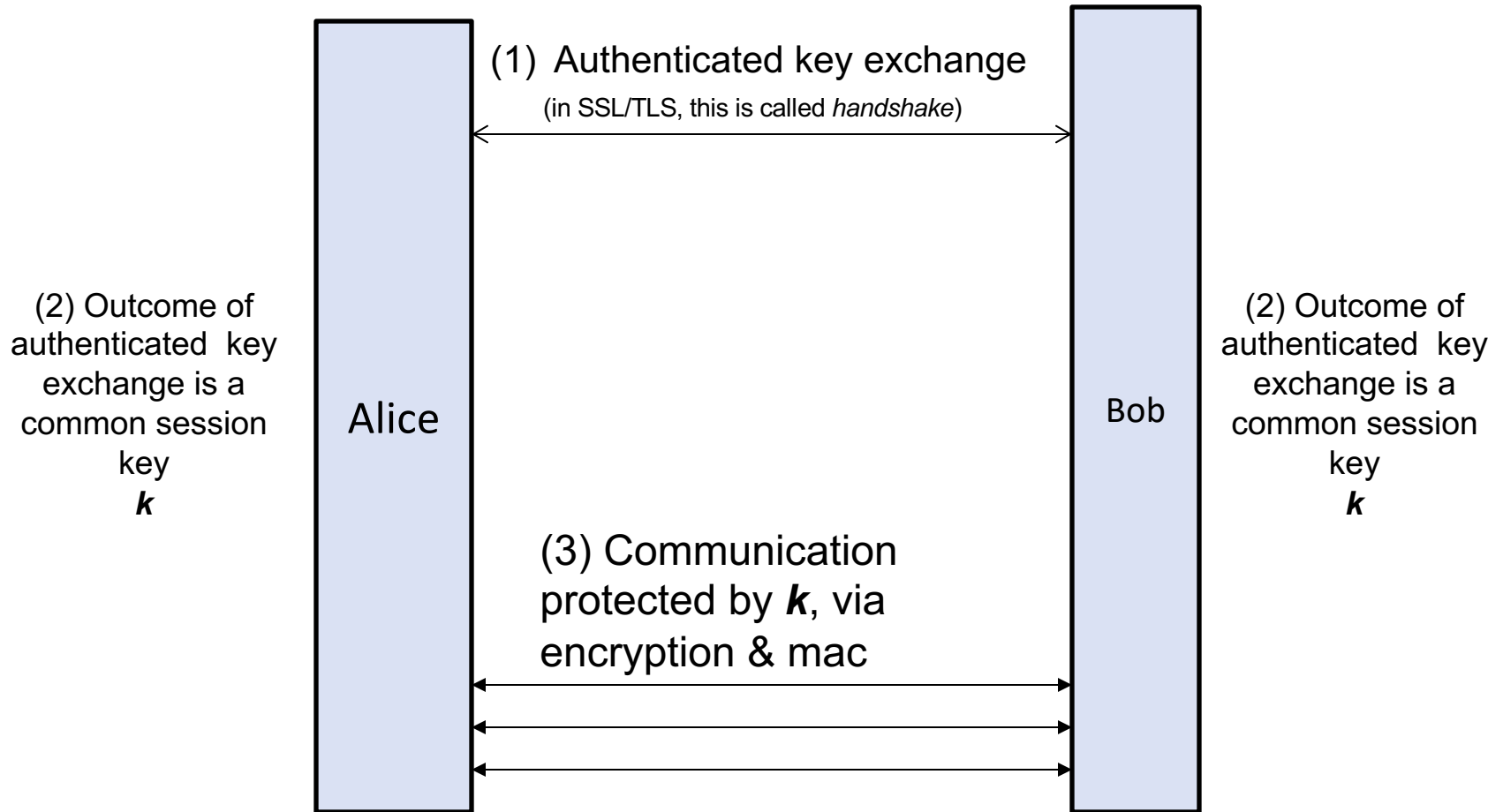
We need something more: Authenticated key-exchange

- You may wonder what come next after authentication. Consider the setting of Alice, Bob and the malicious Mallory. Mallory wants to impersonate Alice. Recall that Mallory is malicious and can sniff, spoof, and modify the message.



Imagine that Mallory allows Alice and Bob to carry out the strong authentication. After Bob is convinced that he is communicating with Alice, Mallory interrupts and takes over the channel. Later Mallory pretends to be Alice! (Here, Mallory is the man-in-the-middle). So, even if we employ the challenge-response in the previous slide, Mallory can still succeed in this setting.

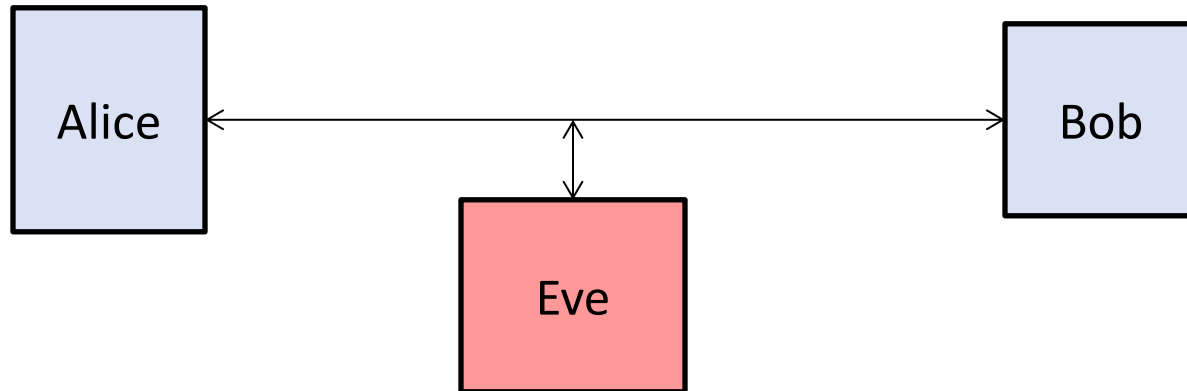
- There is no contradiction here. The previous authentication protocols assume that the adversary is unable to interrupt the session, that is, the entity is the same throughout the session.
- For applications whereby Mallory can interrupt the session, we need something more: The outcome of the authentication process must somehow pass to the next phase. This can be done by establishing a new shared secret ***k***. In other words, the outcome of the authentication protocol is a shared secret ***k***. This shared key is aka ***session key***. Subsequent phase will be protected using ***k***.
- The process of establishing a secret (with or without authentication) between Alice and Bob is called
key-exchange, or key-agreement.
- If the process is incorporated with authentication, then it is called ***Authenticated key-exchange***. A well-known scheme is called ***station-to-station*** protocol.



4.5 Key-exchange, authenticated key-exchange

Key-exchange (Secure against Eve. No authentication)

- Alice and Bob want to establish a common key. The established key can be used to protect (e.g. via cipher, mac) subsequent communication between Alice and Bob.
- The interaction could be eavesdropped by Eve. After seen the interactions, Eve wants to guess the established key.



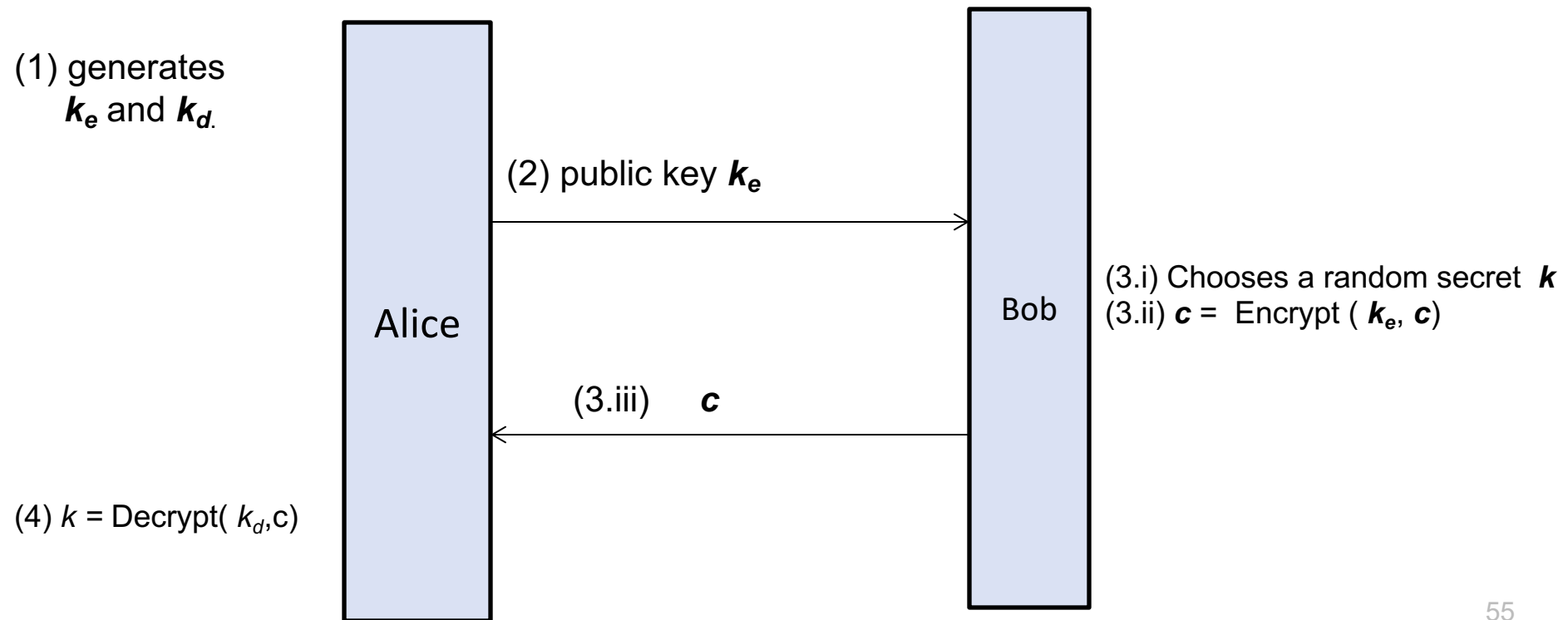
- The next slide gives a method based on PKC. Another well-known method is Diffie-Hellman Key exchange.

*Here, we only consider Eve who can sniff and want to guess the key.
There is no consideration of authenticity here.*

PKC-based Key-exchange (no authentication)

Here is a key-exchange that uses a PKC

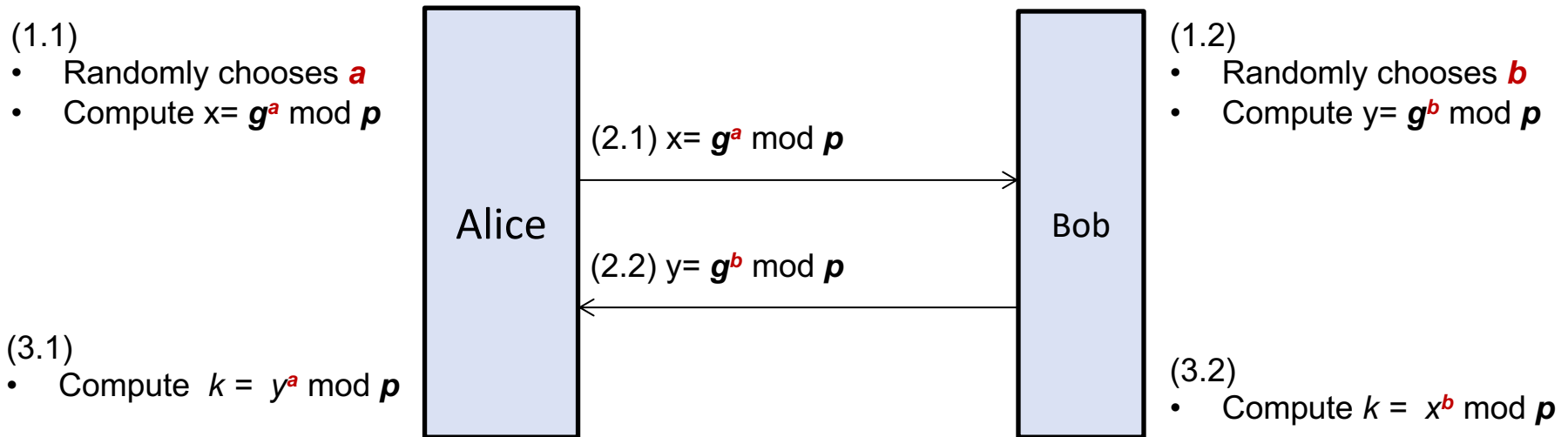
1. Alice generates a pair of private/public key.
2. Alice sends the public key k_e to Bob.
3. Bob carries out the following
 - i. Randomly chooses a secret k ,
 - ii. Encrypts k using k_e .
 - iii. Sends the ciphertext c to Alice.
4. Alice uses her private key k_d to decrypt and obtain k .



- Attacker (Eve) can obtain the public key k_e and the ciphertext c .
- By security of PKC, from the public and ciphertext, attacker can't get any information of the plaintext, which is the key k .

Diffie-Hellman key-exchange (no authentication)

We assume both Alice and Bob have agreed on two *public* parameters, a *generator* g and a large (e.g. 1000 bits) prime p . Both g and p are not secret and known to the public.



Security relies on the CDH assumption.

Computational Diffie-Hellman CDH assumption:

Given g , p , $x = g^a \bmod p$, $y = g^b \bmod p$, it is computationally infeasible to find $k = g^{ab} \bmod p$.

Remarks:

1. Step (1.1)&(1.2), (2.1)&(2.2), (3.1)&(3.2) can be carried out in parallel.
2. The assumption seems self-fulfilling. Nonetheless, there are many evidences that it holds.
3. The operation of “exponentiation” can be applied to any algebraic group, i.e. not necessary integers. CDH doesn't hold in certain groups. The crypto community actively searches for groups that CDH holds. E.g. Elliptic Curve Cryptography ECC is based on elliptic curve group where CDH believed to hold.

Eg.

$$g = 2, \quad p = 23$$

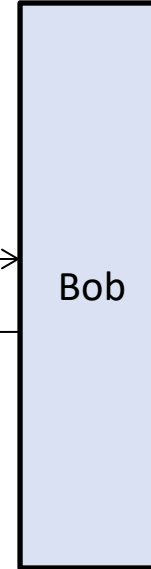
(1.1) randomly
chooses $a = 15$



Alice

$$(2.1) \ x = g^a \bmod p = 16$$

$$(2.2) \ y = g^b \bmod p = 3$$



Bob

(1.2) randomly
chooses $b = 8$

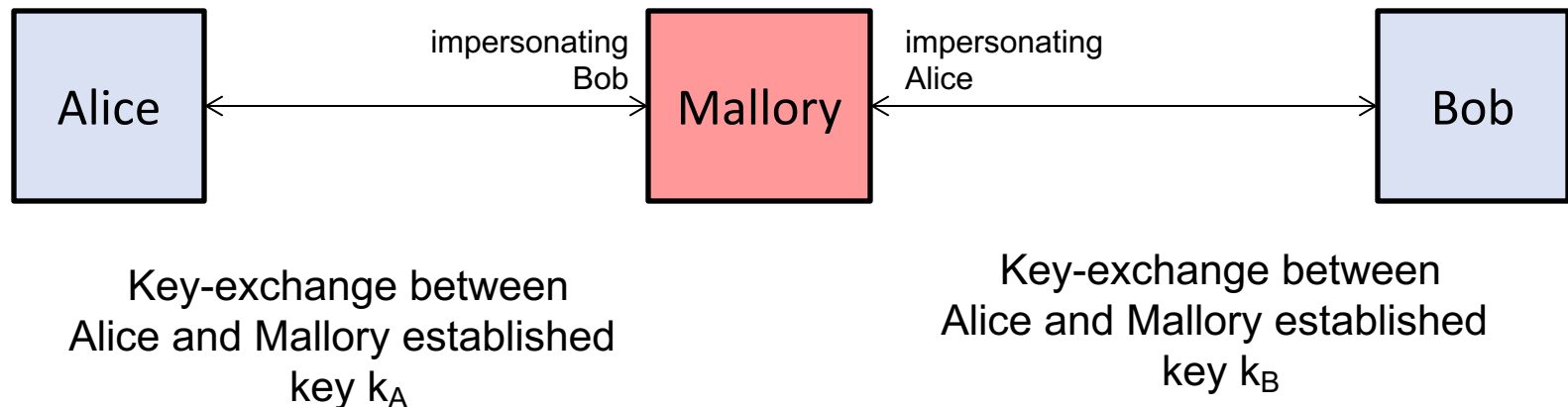
$$\begin{aligned} (3.1) \text{ Compute} \\ k &= y^a \bmod p \\ &= 3^{15} \bmod 23 \\ &= 12 \end{aligned}$$

$$\begin{aligned} (3.2) \text{ Compute} \\ k &= x^b \bmod p \\ &= 16^8 \bmod 23 \\ &= 12 \end{aligned}$$

From 16, and 3, it is very difficult to get 12.
(here, we are referring to very large, say 2000-bit integer)

The basic key-exchange can't guard against Mallory.

- What if the adversary is malicious? Example, a man-in-the-middle?



In this case, Alice mistaken that Mallory is Bob. Communication from Alice is encrypted using k_A . Mallory can decrypt using k_A and re-encrypt using k_B . Hence, Mallory can see and modify the message.

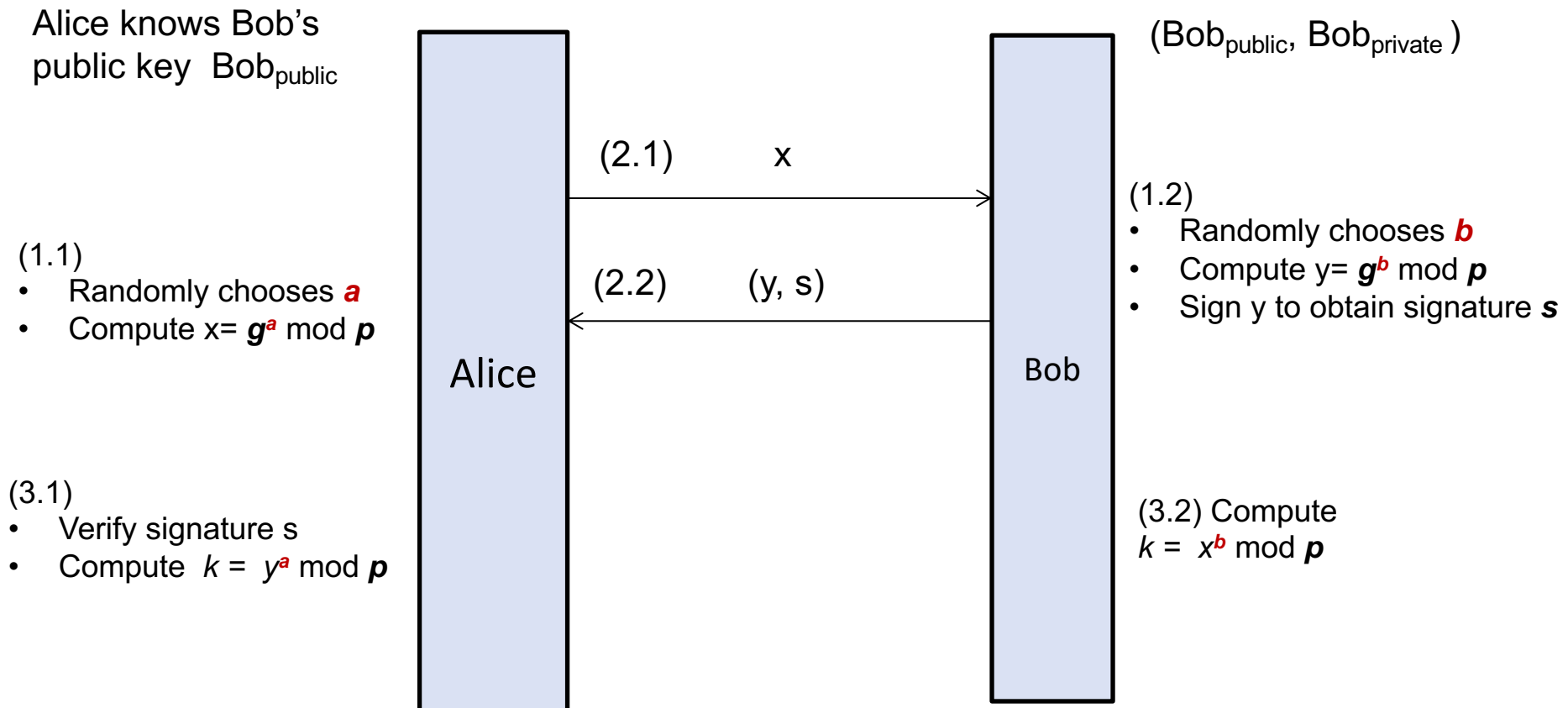
Authenticated key-exchange

- A key-exchange protocol assume that the adversary is can only sniff (in crypto, this is aka curious), but not malicious.
- To prevent malicious Mallory, we need ***authenticated key-exchange***. The authentication can be achieved using PKC. In mutual authentication, Alice and Bob need to know each other public key. In unilateral authentication, only one party need to have public key. After the protocol has completed, a common key (i.e the ***Session key***) is established. Authenticated key-exchange is also applicable in symmetric key setting, although the public key setting is more commonly deployed.
- It turns out that authenticated key-exchange can be easily obtained from existing key-exchange (either PKC-based key-exchange or DH-based key-exchange). This is done by simply signing all communication using the private key.
- We have a special name for authenticated key-exchange that uses DH: it is known as ***Station-To-Station Protocol (STS)***.

Station-to-station protocol (Authenticated key-exchange based on DH)

We assume both Alice and Bob have agreed on two *public* parameters, a *generator* g and a large (e.g. 1000 bits) prime p . Both g and p are not secret and known to the public.

Here, we consider unilateral authentication. Alice want to authenticate Bob.



Remark: This is unilateral authentication. Can extend it to mutual by making Alice signs her messages in step (2.1).

Summary: Mutual Authenticated key exchange

- *Before the protocol:*
 1. Alice has a pair of public, private key (A_{public} , A_{private}).
 2. Bob has a pair of public, private key (B_{public} , B_{private}).
 3. Alice knows Bob public key and vice versa. These two sets of keys are known as the **Long-term key** or **Master key**.
- They carry out Authenticated key exchange protocol (e.g. STS). If an entity is not authentic, the other will halt.
- *After the protocol:*
 1. Both A and B obtain a shared key **k**, known as the **Session key**.
- Security Requirement.
 1. (Authenticity) Alice is assured that she is communicating with an entity who knows B_{private} .
 2. (Authenticity) Bob is assured that he is communicating with an entity who knows A_{private} .
 3. (confidentiality) Attacker unable to get the session key.

4.6 Putting all together: Securing Communication Channel

E.g. TLS

Consider a communication channel that is subjected to a Mallory (sniffing, spoofing, modifying), how to secure it using cryptographic tools?

E.g. Alice wants to visit a website `Bob.com`, how to achieve authenticity (i.e. Alice being assured that `Bob.com` is authentic) and confidentiality (i.e. no info leakage of the communication) ?

In TLS/SSL: (https uses TLS)

- (1) Using ***long-term keys (i.e. Bob's public and private key)***, carry out authenticated key-exchange (aka handshake in TLS) to establish ***session keys***.
- (2) Subsequent communication protected by the session keys.

See more details of TLS handshake in later slides.

https://www.ibm.com/support/knowledgecenter/en/SSFKSJ_7.1.0/com.ibm.mq.doc/sy10660_.htm

Alice wants to visit Bob.com: How TLS does it.

(Step 0) Alice obtains Bob.com's public key. This is done by having Bob sending his certificate to Alice.

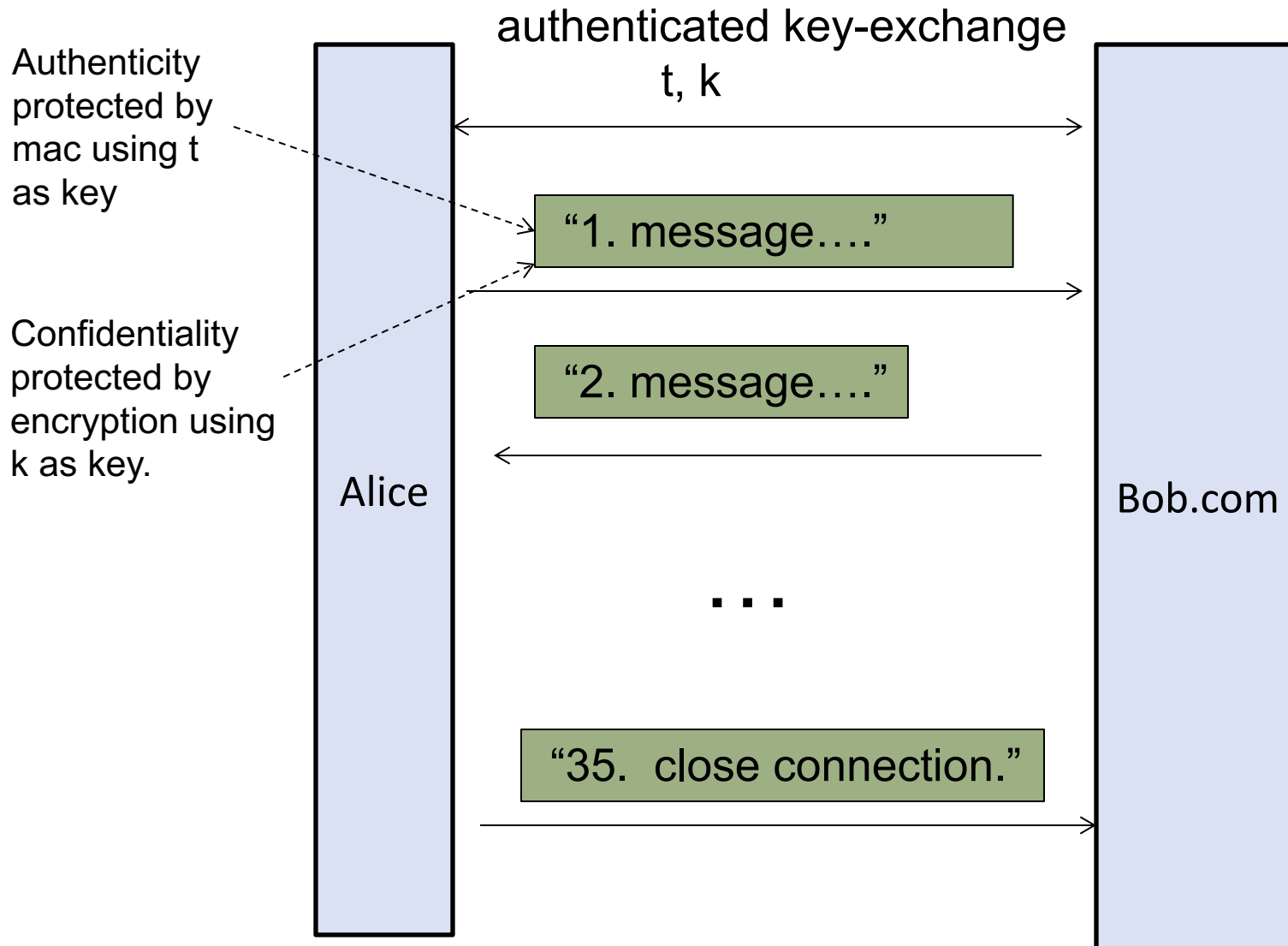
(Step 1) Alice and Bob.com carry out **unilateral authenticated key exchange** protocol with Bob's private/public key. After the protocol, both Bob and Alice obtain two shared keys t, k where t is the secret key of the MAC, and k is the secret key of the symmetric-key encryption, say AES. They are called the **session keys**. From Alice's point of view, the protocol is secure in the sense that, only an entity who knows Bob's private key is able to complete the protocol. So, Alice is convinced that the entity who now holds t, k is Bob. Here, Bob doesn't care about Alice's authenticity.

(Step 2) Subsequent interactions between Alice and Bob.com will be protected by t, k and a sequence number. Suppose $m_1, m_2, m_3, \dots$ are the sequence of message exchanged, the actual data to be sent for m_i is

$$E_k(i \parallel m) \parallel \text{mac}_t(E_k(i \parallel m))$$

where i is the sequence number.

- $\parallel$ refer to string concatenation.
- (optional) The above is known as “encrypt-then-mac”. There are other variants: “mac-then-encrypt” and “mac-and-encrypt”. Using the wrong variant might leak info. When in doubt, use “authenticated encryption” such as AES GCM mode. GCM is fairly new and was not established when TLS was designed.



The data eventually sent is $E_k ("1. \text{ message etc} ") \parallel \text{mac}_t (E_k ("1. \text{ message etc} "))$

Question: What is the role of the sequence number 1,2,3, ... and the last message "close connection"?

Relationship among TLS/SSL/https

- SSL and Transport Layer Security (**TLS**) are protocols that secure communication using cryptographic mean.
- SSL is the predecessor of TLS.
- Https is built on top of TLS.

Question: Alice is in a café. She uses the free wifi to upload her assignment to LumiNUS (which uses https). The café owner controls the wifi router, and thus can inspect every packet going through the network. Can the café owner see Alice's report? **No**. We would discuss this later in network security.

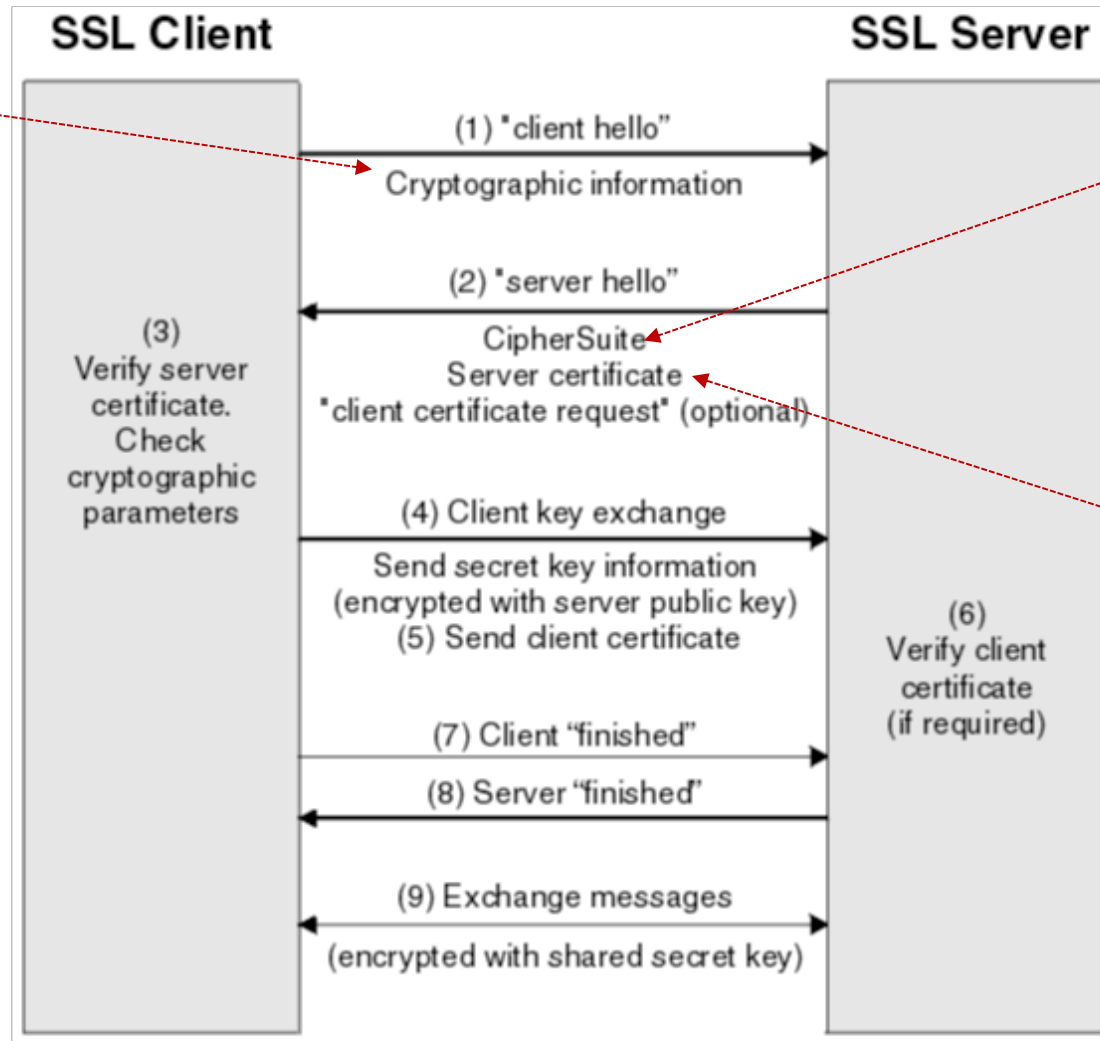
Remarks:

- SSL 3.0 has a number of known vulnerabilities in its crypto implementation. Vulnerable to padding oracle attack, etc. (e.g CVE-2014-8730). TLS 1.0 is an upgraded version of SSL3.0 (in 1999). But TLS 1.0 allows fallback to SSL3.0 (aka downgrading attack), and thus TLS 1.0 is also vulnerable. So, to be secure, at least TLS 2.0. See https://en.wikipedia.org/wiki/Transport_Layer_Security#TLS_1.0

TLS handshake (authenticated key exchange)

Negotiation of the type of crypto to be used, e.g. whether it is STS, RSA, and the key length. Here, Client propose the suite to be used.

Authenticated key-exchange



Server confirms or counter-proposes the crypto suite.

Server's public key